

Smoker: Architecture and Execution Model

A Docker-based framework for reproducible Perl module compatibility testing

1. Introduction

Smoker tests Perl/CPAN modules under controlled dependency configurations. It executes each unique plan row in an isolated Docker container, preserves the factual evidence for that execution, and writes a compact summary row for batch analysis.

Smoker separates two questions that are easy to confuse:

- What happened to the tested module or dependency configuration?
- Did Smoker itself execute the row and preserve the required evidence?

Module outcomes are reported as PASS, FAIL, or UNAVAILABLE. Independent validation determines whether the batch evidence satisfies the execution invariant.

2. Test Modes

Smoker supports three principal modes.

baseline Tests the target module without selecting dependency versions.

vary-one Selects and verifies one dependency version before testing the target.

vary-two Selects and verifies two dependency versions before testing the target.

3. Execution Flow

Run_Smoker.pl prepares the batch, preserves the requested plan, deduplicates plan rows using parsed CSV field identity, and invokes the scheduler. After scheduling completes, Run_Smoker.pl runs the independent validator and uses its result to decide whether the batch evidence is acceptable.

The scheduler parent controls concurrency. Before a worker starts, the parent allocates and reserves the run directory, records the association among plan row, worker PID, and run directory, and passes that exact assigned directory to the worker.

Runner executes one assigned row. It uses the assigned run directory, invokes Docker, classifies the module outcome, and finalizes the required artifacts. Catchable Runner exceptions after a run directory exists are finalized as evidenced framework failures.

If a scheduled worker cannot finalize its assigned row, the scheduler parent uses its retained mapping to recover the row as an evidenced framework failure. Scheduler-parent crash and restart-time abandoned-batch recovery are not currently supported.

4. Plan Identity

The input plan has this eight-column header:

```
base,mode,module,version,dep_one,dep_one_version,dep_two,dep_two_version
```

Duplicate identity is based on the complete ordered set of parsed CSV fields. Equivalent valid CSV serializations, such as optional quoted versus unquoted fields, describe the same plan row. Rows with genuinely different parsed field values remain distinct. For vary-two rows, the two dependency-version tuples form an unordered pair, so reversing the pair does not create another unique row. Blank lines and comment lines are preserved by the launcher and are not counted as plan rows.

5. Evidence And Result Artifacts

For each unique plan row, Smoker preserves a run directory containing factual execution evidence. Required result artifacts include:

```
rc.code  
result.meta  
execution_trail.audit
```

Framework failures also require explicit framework-error evidence, such as `framework_error.evidence`. Producers may record facts such as commands, stages, timestamps, exceptions, raw wait status, signals, and recovery actions. They do not certify that their own output is complete or correct.

6. Summary Data

`summary.csv` uses this seventeen-column schema:

```
run_id  
batch  
base  
mode  
module  
version  
dep_one  
dep_one_version  
dep_two  
dep_two_version  
status  
rc  
start_ts  
end_ts  
elapsed_s  
run_dir  
note
```

The stable summary lock also protects a batch-local run-id membership index. Legacy summaries bootstrap that index with one complete scan. Ordinary first-time rows then append

without parsing earlier rows. Existing index membership catches lost or empty per-run state, while pending or damaged state also selects atomic full-file reconciliation for retry and recovery. After an append, the transaction remains pending until its index entry and committed state have both been written, so a crash at any earlier point is reconciled idempotently.

Launcher shutdown gives scheduler process groups five seconds after TERM or INT before escalating to KILL. It then blocks to reap the owned group leader rather than deliberately leaving a child unmanaged. A process stuck in uninterruptible kernel sleep can therefore extend the final reap beyond the escalation bound.

Requested dependency versions are compared using Perl version semantics. If a module exposes no package VERSION, Smoker logs that local verification is inconclusive and continues; this is not proof of an exact installed version.

7. Result Policy

Public Smoker result policy is:

```
PASS: rc=0
UNAVAILABLE: rc=111
framework failure: status=FAIL, rc=125, subtype=framework_error
```

Confirmed unavailability is emitted as rc=111 by the current Inner layer. The shared result policy also accepts rc=4 with explicit confirmation evidence and normalizes it to rc=111; an unconfirmed rc=4 remains FAIL,4. A public bare rc=125 is invalid without subtype=framework_error and explicit framework evidence. Ordinary module failures remain FAIL results and are not framework failures.

8. Independent Validation

bin/validate_smoker_batch.pl independently derives batch integrity conclusions. It checks expected execution paths, required result files, result classification, summary-row uniqueness, run-directory association, and framework-failure evidence. Producer-authored claims such as PASS, complete, verified, or integrity satisfied do not override contradictory artifacts.

The user-facing validator headings are:

```
Expected execution paths
  Satisfied
  Unsatisfied

Required result files
  Complete
  Incomplete

Result classification
  Classified
  Unclassified
```

9. Backup And Optional MiniCPAN

Run_Smoker.pl owns the pre-run backup decision and asks:

```
Run pre-test backup? [y/N]
```

Answer y or Y to run the backup. Enter, n, N, invalid input, timeout, EOF, and noninteractive execution skip it. This default favors fast troubleshooting runs. bin/bksmoker.sh performs an explicitly requested backup without prompting.

MiniCPAN remains optional. The normal source backup does not require rsync. For backups, rsync is required only when SMOKER_BACKUP_MINICPAN explicitly requests copying the MiniCPAN mirror. The separate maintainer tree-sync utility also requires rsync.

10. Large-Run Validation And Forensic Audit

Smoker has been exercised in a 61,035-case test campaign across five Perl versions. The historical batch contained 18,398 PASS, 31,077 FAIL, and 11,560 UNAVAILABLE results.

Because the unexpectedly large FAIL count could not safely be interpreted as 31,077 reproducible module failures, the original results were preserved and subjected to a subsequent forensic audit using targeted confirmation runs and causal recalibration.

The audit distinguishes the historical execution result from its later interpretation. Historical rows are not rewritten. Instead, subsequent evidence records whether a failure mechanism reproduced, proved version-dependent, represented an unavailable exact release, changed mechanism, or did not reproduce under confirmation conditions.

All historical failure rows now have a causal recalibration assignment. Evidence strength is recorded explicitly: some conclusions are based on direct confirmation, some on exact module/version pair evidence across five Perl bases, and the historical dependency-did-not-remain population remains sample-supported rather than individually rerun.

See `Smoker_61035_Final_Forensic_Audit_Report.md` for the methodology, evidence buckets, confidence boundaries, and reproducible audit artifacts.

11. Conclusion

Smoker is organized around this execution invariant:

```
One unique plan row -> one run directory -> one rc.code -> one summary row
```

The invariant is supported by factual producer artifacts and enforced by independent validation, not by producer self-certification.

That invariant establishes the integrity of the recorded execution. It does not, by itself, establish the ultimate causal interpretation of a module result. Later confirmation evidence may refine that interpretation while leaving the historical result immutable.